Differential-Drive Robot with the CodeCell C6 Drive

Design, Control & Wireless Operation

Krzysztof Sawicki, Michał Szerechowicz, Mateusz Skorupiński

Poznań University of Technology
Automation Control and Robotics

June 13, 2026

Outline

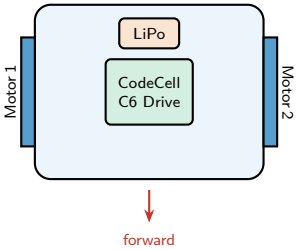
1. Project Overview
2. Hardware
3. Software Architecture
4. Movement Control
5. Wireless Communication
6. Control Application
7. Safety & Robustness
8. Summary & Future Work

Project Goals

Build a **compact, wirelessly controlled** differential-drive robot using the CodeCell C6 Drive as the sole electronic module.

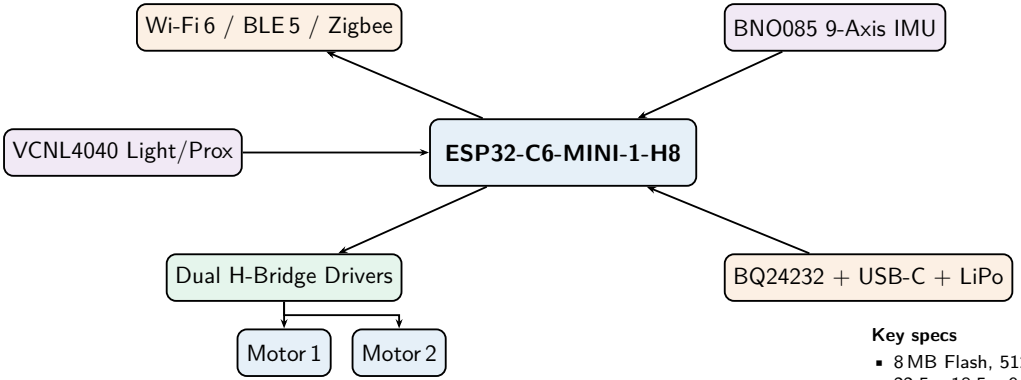
Key objectives:

- Heading-stabilised motion via onboard IMU
- Two operating modes: real-time joystick & autonomous point-to-point
- Browser-based control app (no install needed)
- 3D-printed chassis – reproducible build



Top-down schematic view

CodeCell C6 Drive – Board Overview



Key specs

- 8 MB Flash, 512 KB SRAM
- $22.5 \times 18.5 \times 9.4$ mm
- 3.4 g (board only)
- DRV8837 H-Bridge per channel
- Sensor-fusion on BNO085

Mechanical Design

Differential-drive layout

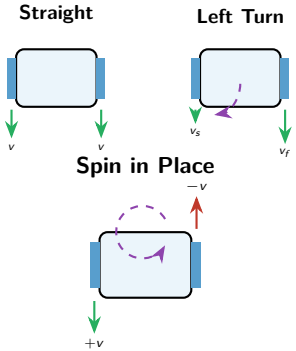
- Left motor → left wheel
- Right motor → right wheel
- Board and LiPo mounted centrally

Chassis

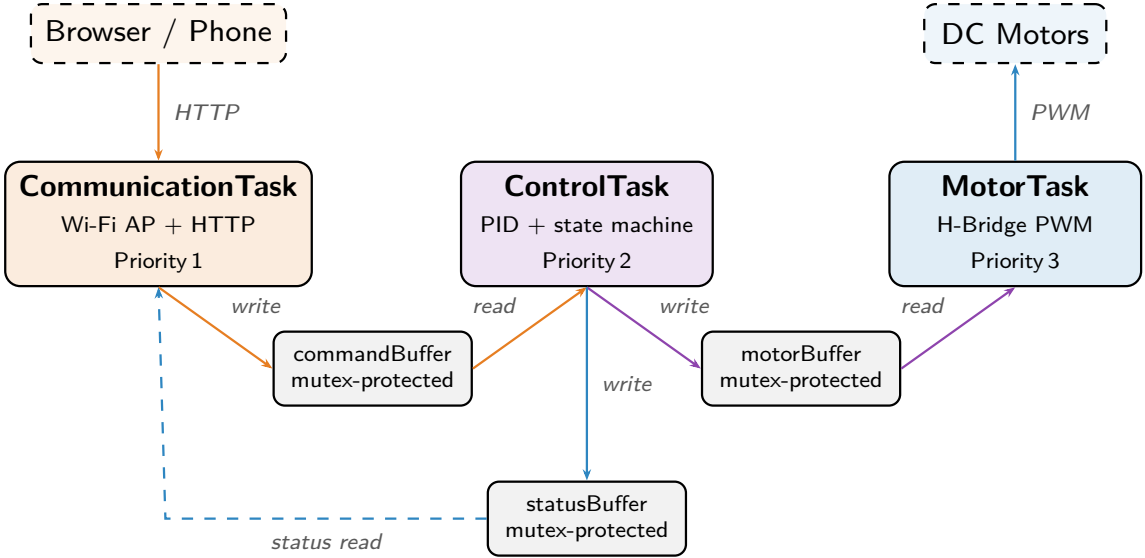
- 3D-printed (FDM), single-piece shell
- Designed for press-fit motor mounting

Turning principle

Equal speed ⇒ straight line
Speed difference ⇒ curved path
Opposite speeds ⇒ rotation in place



FreeRTOS Task Architecture



Differential-Drive Motor Mixing

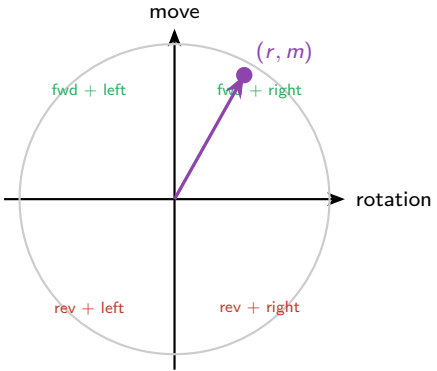
Joystick inputs in $[-100, 100]$:

- `move`: forward/backward
- `rotation`: turning

Mixing equations

$$m_1 = \text{clip}(\texttt{move} + \texttt{rot})$$
$$m_2 = \text{clip}(\texttt{move} - \texttt{rot})$$

where $\text{clip}(x) \in [-100, 100]$. Sign sets direction; magnitude sets PWM duty cycle.



Example

move $= 70$, rot $= 30$

$m_1 = 70 + 30 = 100$

$m_2 = 70 - 30 = 40$

$\Rightarrow$ gentle right curve

Heading-Stabilised Control – PID on Yaw

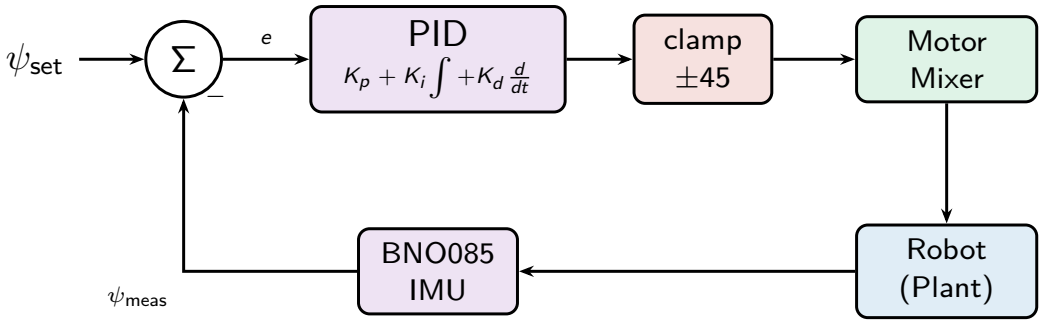
The BNO085 provides fused **yaw** (heading) at each control cycle ($\sim$50 Hz). A discrete PID controller minimises heading error and feeds its output as the `rotation` input to the motor mixer.

Tuning parameters

Parameter	Value
K_p	0.30
K_i	0.02
K_d	0.02
Integral limit	± 120
Max correction	± 45

Error is wrapped to $(-180^\circ, 180^\circ]$ so the controller always uses the shortest rotation path.

PID Yaw Feedback Loop



Control Modes – Direct vs. Target

Both modes are selected by the REST API command sent to the robot.

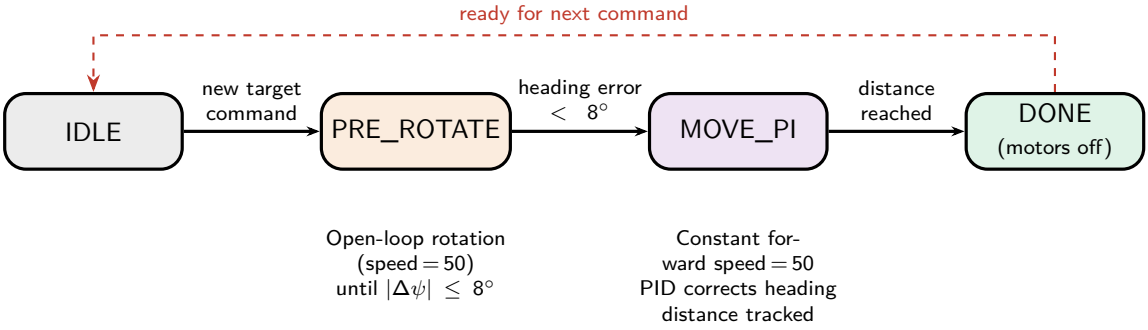
Direct Mode (Joystick)

- Joystick sends (`move`, `rotation`) at 10 Hz via HTTP
- `rotation` $\rightarrow$ yaw rate (degrees/s scaling)
- PID maintains heading setpoint
- Stick release $\rightarrow$ auto-centre, sends $(0, 0) \rightarrow$ stop

Target Mode (Autonomous)

- User specifies distance (m) and angle ($^\circ$)
- Phase 1: rotate to target heading
- Phase 2: drive straight with PID yaw correction
- Distance by timer *or* IMU

Target Mode – Two-Phase Execution



Distance estimation: time-based calibration or IMU double-integration.

Start Ramp & Motor Deadband Compensation

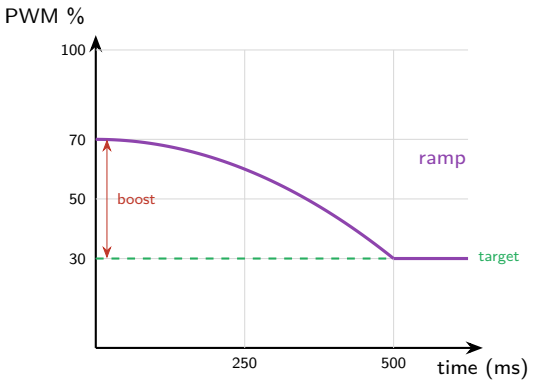
Small DC motors need extra torque to overcome **static friction** at startup.

Start-ramp algorithm

$$v_{\text{out}}(t) = |v_{\text{target}}| + (v_{\text{boost}} - |v_{\text{target}}|) \cdot (1 - t^2)$$

where $v_{\text{boost}} = 70\%$, ramp duration $T = 500\,\text{ms}$, and $t = \text{elapsed}/T$.

The output is clamped to 100 and returns to the requested speed after 500 ms.



Wi-Fi Access Point & REST API

The robot creates its own **Wi-Fi hotspot**; the browser connects directly.

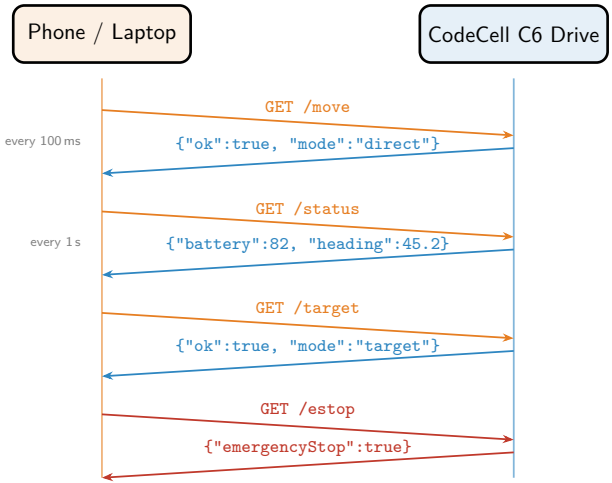
Setting	Value
SSID	`CodeCell-C6-Drive`
Password	`12345678`
IP	`192.168.4.1`
Protocol	HTTP (port 80)

REST commands use HTTP `GET` requests and JSON responses.

API Endpoints

Endpoint	Purpose
`/move`	Joystick speed command
`/target`	Distance + angle command
`/status`	Battery, heading, motor state
`/estop`	Emergency stop
`/clear_estop`	Clear e-stop

Communication Flow



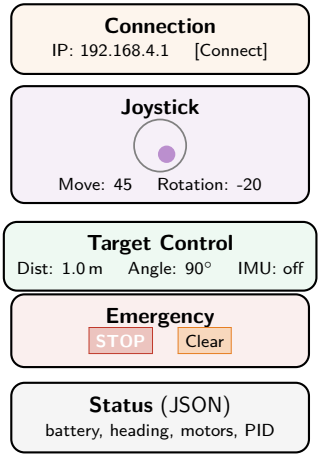
Web-Based Control Interface

Single-page HTML/JS app

- Runs in any modern browser
- Touch-friendly virtual joystick
- Mouse and touch pointer events

Features

- Joystick sends `/move` at 10 Hz
- Target panel: distance, angle, IMU toggle
- Emergency stop / clear buttons
- Live status refreshed every second



Safety Mechanisms

Emergency Stop

- One-click E-Stop via `/estop`
- All tasks check one shared flag
- Cleared only by explicit `/clear_estop`

Command Timeout

- Every command is timestamped
- No update within **300 ms** $\rightarrow$ stop
- Handles Wi-Fi or browser failure

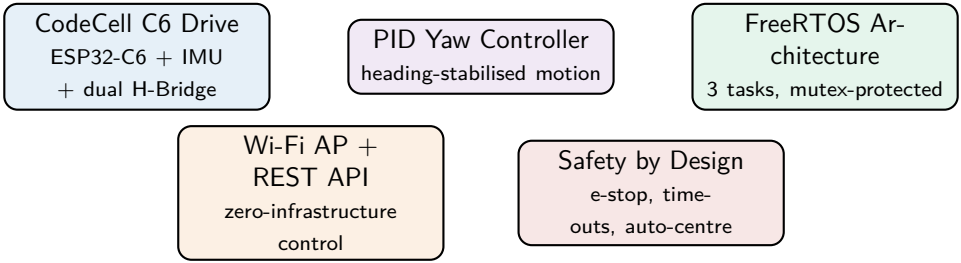
Thread Safety

- Three FreeRTOS mutexes guard shared buffers
- No direct cross-task memory access
- Sequence counter detects new commands

Auto-Centre Joystick

- On `pointerup`, joystick snaps to $(0, 0)$
- Sends stop command immediately
- Prevents unintended continued motion

Summary



A self-contained differential-drive robot built from one compact board, controlled from a browser, with real-time heading stabilisation from the onboard 9-axis IMU.

Future Work

- **Obstacle avoidance** – reactive stop / turn behaviours
- **Odometry improvements** – wheel encoders or visual odometry
- **Multi-waypoint navigation** – sequence of target commands
- **BLE / Zigbee control** – lower-power communication options
- **Smartphone app** – richer UI and persistent settings

Thank you

Questions?

Source code & 3D models available in the project repository